nixlab – Projektdokumentation

Stand: 19.10.2026

> Diese Dokumentation beschreibt ein privates Repo. Domain und interne Adressen sind durch Beispielwerte ersetzt: die Domain durch `homelab.example` (RFC 2606), IPv4 nach RFC 5737 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`), IPv6 nach RFC 3849 (`2001:db8::/32`), der Tailnet-Name durch `example-tailnet.ts.net`. Die echte Mesh-Range ist ein /22 aus RFC 1918. Module, Dokumentation und Build-Werkzeuge sind zusammen mit Claude (Anthropic) entstanden. Lizenz: MIT.

Überblick

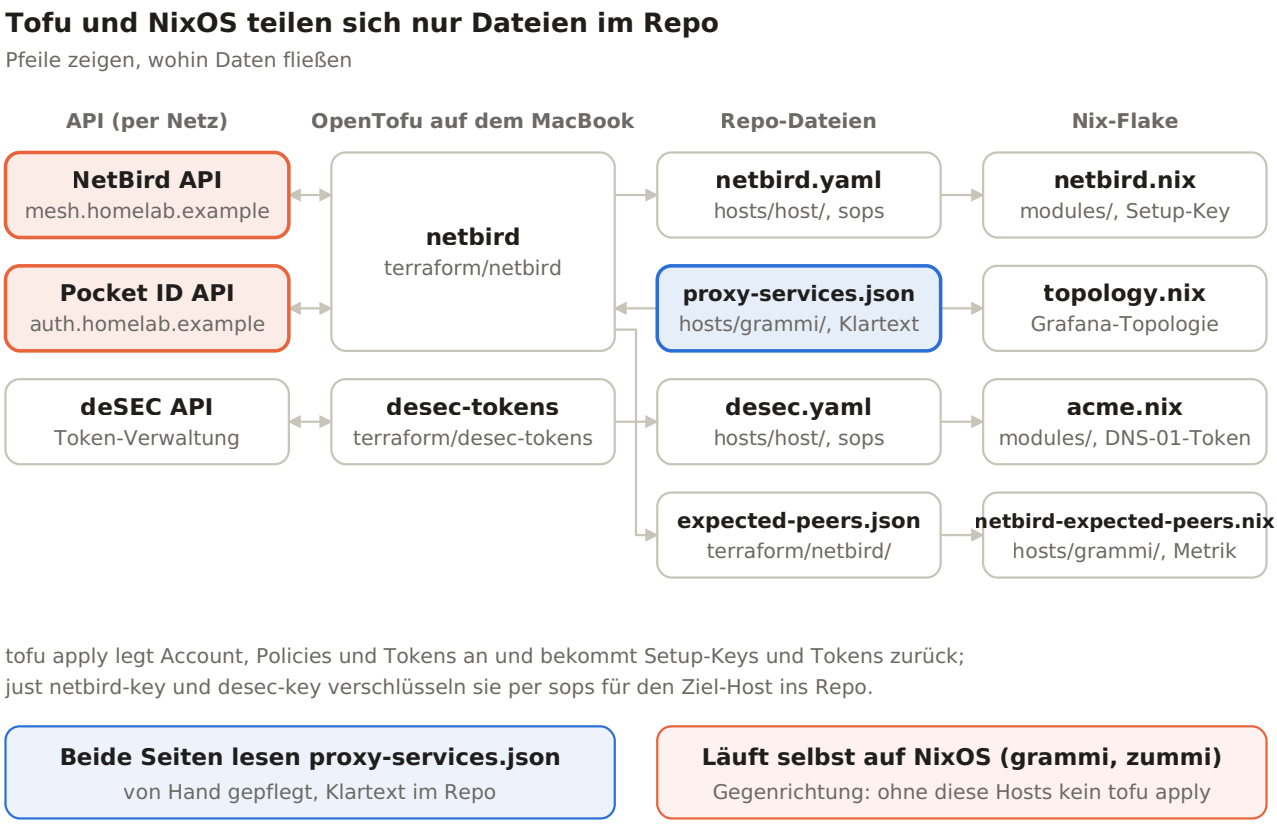
nixlab ist das Flake-Monorepo für das Homelab **homelab.example**: fünf NixOS-Hosts und ein Mac mini mit nix-darwin, dazu drei OpenTofu-Module für den NetBird-Account, die deSEC-API-Tokens und die GitHub-Repos samt Deploy-Keys. Bedient wird alles über ein `justfile` vom MacBook aus; die Hosts ziehen ihre Konfiguration selbst aus dem privaten GitHub-Repo und aktualisieren sich nachts.

Kurzantwort auf die Frage *„Hast du NixOS und OpenTofu irgendwie miteinander verdrahtet?“*: **Ja, aber bewusst lose – über Dateien im Repo, nicht über ein gemeinsames Tool.** OpenTofu schreibt sops-verschlüsselte Dateien in `hosts/<host>/`, die NixOS-Module beim Build lesen, und beide Seiten lesen dieselbe Dienstliste `hosts/grammi/proxy-services.json`. Kein terranix, kein Tofu-Lauf aus Nix heraus. Details unter NixOS & OpenTofu.

Zielgruppe: Leute, die NixOS-Flakes, sops-nix und Terraform/OpenTofu lesen können.

NixOS & OpenTofu

Ja, beides ist verdrahtet – über das Repo als gemeinsame Schnittstelle. OpenTofu verwaltet das, was außerhalb der Maschinen per API existiert (NetBird-Account, deSEC-Tokens, Pocket-ID-Gruppen); NixOS verwaltet die Maschinen. Die beiden reden nie direkt miteinander: Tofu läuft nur auf dem MacBook über `just`, Nix wertet nie Tofu-State aus.



Tofu schreibt nie direkt auf einen Host: jedes Ergebnis landet als Datei im Repo und erreicht den Host erst mit dem nächsten Deploy.

Die Berührungspunkte

Richtung	Datei	Erzeugt von	Gelesen von	Inhalt
Tofu → NixOS	`hosts/<host>/netbird.yaml`	`just netbird-key <host>` (`tofu output setup_keys` → `sops encrypt`)	`modules/netbird.nix` (`sopsFile`)	Einmal-Setup-Key, 24 h gültig
Tofu → NixOS	`hosts/<host>/desec.yaml`	`just desec-key <host>` (`tofu output acme_tokens` → `sops encrypt`)	`modules/acme.nix` (`sopsFile`)	deSEC-Token nur für TXT-Records, VMs auf ihre IP gepinnt
Tofu → NixOS	`terraform/netbird/expected-peers.json`	`just netbird-apply` (`tofu output expected_peers`, aus den Policies in `expected-peers.tf` berechnet)	`hosts/grammi/netbird-expected-peers.nix` (`builtins.readFile`)	Soll-Anzahl NetBird-Peers je Host als Metrik `netbird_expected_peers`, im NetBird-Dashboard neben der echten Network Map (z. B. aldrin 6, grammi 8)
gemeinsam	`hosts/grammi/proxy-services.json`	von Hand	`terraform/netbird/reverse-proxy.tf` (`jsondecode(file(...))`) und `topology.nix`	Name → App, Port, Gruppen der Reverse-Proxy-Dienste

Richtung	Datei	Erzeugt von	Gelesen von	Inhalt
Repo → beide	`hosts/<host>/github.yaml` + `github-deploy.pub`	`just github-key <host>` (`ssh-keygen` → `sops encrypt`)	privat: `modules/deploy.nix` (`sopsFile`); öffentlich: `terraform/github` (`file(...)`)	Deploy-Key für `/etc/nixos`; Tofu sieht nur den öffentlichen Teil
NixOS → Tofu	NetBird-Server auf grammi	`hosts/grammi/netbird.nix`	Provider über `https://mesh.homelab.example`	Die API, gegen die Tofu überhaupt erst läuft
NixOS → Tofu	Bootstrap-Tokens	`just netbird-setup-owner`, `netbird-proxy-token` (SSH auf grammi, Server-API)	`terraform/netbird/secrets.yaml` bzw. `hosts/grammi/netbird-proxy.yaml`	Owner-PAT, Proxy-Token

Der Ablauf für einen neuen Host: Tofu legt den Setup-Key an, `just netbird-key` schreibt ihn verschlüsselt für genau diesen Host (age-Key aus seinem SSH-Host-Key) nach `hosts/<host>/netbird.yaml`, Commit, Deploy innerhalb von 24 h. Der Host registriert sich beim Start selbst; Peers selbst verwaltet Tofu nicht, es findet sie per `data "netbird_peer"` über den Namen.

Was bewusst nicht verdrahtet ist

- **Keine gemeinsame Host-Liste.** Hosts stehen dreimal: `nixosConfigurations` in `flake.nix`, `local.hosts` in den Tofu-Modulen und `hosts :=` im `justfile`. Ein neuer Host muss an allen drei Stellen eingetragen werden.
- **Kein terranix / kein Tofu im Nix-Build.** Tofu-State liegt lokal (gitignored, im Haupt-Checkout, auch aus Worktrees), weil er Setup-Keys und Tokens im Klartext hält.
- **Kein Provisioning der Maschinen.** Die VMs werden beim Anbieter von Hand angelegt, ursalia/cuckoosegg nach `docs/install-*.md`.
- **Öffentliches DNS** liegt in DNSControl (eigenes Repo, nur per Symlink eingebunden), nicht in Tofu.

Die Grenze ist also: Was Nix aus dem Repo heraus deterministisch bauen kann, macht Nix. Was nur einmal per API erzeugt wird und ein Geheimnis zurückgibt, macht Tofu – und übergibt es verschlüsselt als Datei.

Secrets, Netzwerk & Zugriff

Secrets (sops-nix, age)

Jede Host-Datei ist für genau zwei Empfänger verschlüsselt: den age-Key aus dem SSH-Host-Key des Hosts und den Offline-Recovery-Key `admin`. Das MacBook kann Host-Secrets also nicht lesen – `just secrets <host>` editiert sie auf dem Host und holt das Ergebnis zurück. Verschlüsseln für einen Host geht dagegen überall (nur der öffentliche Key ist nötig); genau das nutzen `netbird-key` und `desec-key`.

Datei	Empfänger	Inhalt
`hosts/<host>/secrets.yaml`	Host + admin	Deploy-Key, App-Passwörter, OIDC-Clients, restic
`hosts/<host>/netbird.yaml`	Host + admin	NetBird-Setup-Key (von Tofu)
`hosts/<host>/desec.yaml`	Host + admin	deSEC-Token (von Tofu)
`hosts/<host>/github.yaml`	Host + admin	GitHub-Deploy-Key (von `just github-key`)
`terraform/*/secrets.yaml`	workstation + admin	API-Tokens für die Provider

Der Workstation-Key liegt in 1Password und wird über `SOPS_AGE_KEY_CMD` gelesen, nie als Datei. `env.sh` in jedem Tofu-Modul entschlüsselt die Tokens nur in die Shell des jeweiligen Rezepts.

Netzwerk

- **NetBird-Mesh** (`192.0.2.0/24`, Peer-Namen unter `peer.homelab.example`): Control Plane auf grammi, Login über Pocket ID. Private Range (RFC 1918) statt CGNAT, weil tailscaled Pakete aus 100.64/10 verwirft, solange beide Clients parallel laufen.
- **Gruppen:** `servers` (alle Hosts außer cuckoosegg), `subraum-devices`, `admin-workstations` (MacBook), dazu Personengruppen aus Pocket ID.
- **Heim-LAN** (`198.51.100.0/24`): Route über ursalia (Metrik 100), die beiden Apple TVs als Rückfall (Gruppe `home-routers`, Metrik 200, eigener Setup-Key per `just netbird-router-key`, keiner Person zugeordnet, in keiner Policy); für Gruppe `home-lan`, nicht für Geräte, die schon zu Hause sind (Posture-Check `not-at-home`). Die Apple TVs brauchen in der App „Quantum-Resistance“, sonst nimmt kein Peer (Rosenpass strikt) einen Tunnel zu ihnen an.
- **Anbindung von ursalia:** WLAN ins Hauptnetz des Heim-LANs (Intel BE200, `hosts/ursalia/wifi.nix`): wpa_supplicant, WPA3 mit WPA2-Rückfall, Land DE, alle Bänder, Passwort per sops. Am endgültigen Platz gibt es kein Kabel; eno1 behält DHCP als Rückfall mit schlechterer Routen-Metrik, damit das WLAN die Hauptverbindung bleibt, wenn beide aktiv sind.
- **Subraum:** Route über cuckoosegg für `subraum-access`. cuckoosegg selbst darf nur grammis Ingest (TCP 8428/9428) erreichen und nur vom MacBook per SSH erreicht werden.
- **Tailscale** läuft während der Migration noch parallel und dient als Break-Glass: `NIXLAB_DOMAIN=example-tailnet.ts.net just deploy zummi`.

Zugriff

Öffentlich offen sind nur 22/80/443. root-SSH lässt `modules/ssh.nix` nur von Mesh- oder Tailscale-Adressen zu; jeder Host hat ein eigenes Login-Keypair. Das Mesh ist derzeit nur IPv4, Dual-Stack ist als offener Punkt notiert.

Deployment & Betrieb

Die Pull-Hosts (zummi, aldrin, grammi, ursalia) halten `/etc/nixos` als Checkout des privaten GitHub-Repos und bauen sich selbst. cuckoosegg wird gepusht, zorlock lokal per `darwin-rebuild` umgestellt.

Host	Manueller Deploy	Automatisch
zummi, aldrin, grammi, ursalia	`just ship <host> "<Nachricht>"` = commit, push, `git pull` + `nixos-rebuild switch` auf dem Host	nächtliches Auto-Upgrade, Reboot-Fenster 04:00–06:00
cuckoosegg	`just deploy-push cuckoosegg` (gebaut auf ursalia, kopiert zum Ziel)	ursalia pusht täglich um 06:30
zorlock	`just deploy-zorlock` (`darwin-rebuild switch` mit sudo)	keins

Nächtliches Upgrade

`modules/auto-upgrade-git.nix` aktualisiert `nixpkgs`, `nixpkgs-unstable` und `sops-nix`, baut, schaltet um und pusht die neue `flake.lock` mit dem Deploy-Key des Hosts zurück nach GitHub. Der GitHub-Host-Key ist gepinnt, weil niemand zusieht. Ein Konfigurationsfehler, der erst beim Kaltstart auffällt, legt den Host also um 04:00 lahm – bei den VMs gibt es dafür die Konsole des Anbieters.

Monitoring

Alles landet auf grammi: **vmagent** auf jedem Host schickt Metriken an VictoriaMetrics (`:8428`), **Vector** journald- und nginx-Logs an VictoriaLogs (`:9428`), beides über das Mesh. Grafana (`metrics`) und logchef (`logs`) liegen hinter dem Reverse Proxy mit SSO. Die Exporter je Host kommen aus `modules/metrics.nix`; was keinen eigenen Endpunkt hat, geht über den textfile-Collector von node_exporter (`/var/lib/prometheus-node-exporter-text-files`), geschrieben von kleinen Timern oder Diensten:

Textfile-Metrik	Host	Quelle
`tailscale_*`	Hosts mit Tailscale	`tailscale metrics write`, alle 30 s
`fwupd_updates_*`	ursalia	`fwupdmgr get-updates` nach jedem Metadaten-Refresh
`wifi_link_*`	ursalia	`iw dev wlp2s0f0 link` (PHY-Modus, Breite, MCS, NSS), alle 30 s
`netbird_expected_peers`	grammi	`terraform/netbird/expected-peers.json`, beim Bauen ins System übernommen

Die Dashboards liegen als JSON in `hosts/grammi/dashboards/` und werden aus dem Repo provisioniert (Änderungen nur per Commit + Deploy von grammi). `topology.json` erzeugt `just topology`.

Dashboard	Zeigt
Hosts Overview	Scrape-Ziele, CPU/RAM/Platte/Netz je Host, fehlgeschlagene Units, Zertifikate, Backups (btrbk, restic), ausstehende Firmware, WLAN-Band/-Signal
Topology	Hosts, Exporter und Datenflüsse als Graphviz-Diagramm
NetBird	Erreichbarkeit der Control Plane je Host; Peers je Host als Soll (Tofu) / known (Network Map) / connected P2P / connected Relay, dazu Abweichungen; Verlauf der Verbindungen, P2P-Latenz, Sync- und Aufbauzeiten; netbird-server (Signal, Relay, gRPC, API)
WLAN	Band von ursalia (2,4 / 5 / 6 GHz) über die Zeit, Kanal, Signal, PHY-Modus, Linkrate, Verkehr, MCS, Retries, Verbindungsdauer
Disk Health	SMART/NVMe von ursalia: Status, kritische Warnungen, Verschleiß, Reserve, Temperatur, geschriebene Daten
Paperless	Zustand (Celery, Redis, DB, Index, Classifier), Dokumente, Inbox, fehlgeschlagene Tasks, Speicher, Log-Warnungen
paperless-gpt	OCR- und Vorschlags-Rückstau, OCR-Zeit je Seite, LLM-Anfragen, -Dauern, -Tokens, -Fehler; GPU und Speicher von zorlock
Web & Security	Anfragen an nginx und Traefik, Statuscodes, Dauern; CrowdSec-Bans, -Alerts, Parser-Treffer
Garage (S3 storage)	Garage auf aldrin: Cluster-Zustand, Platte, S3-Anfragen/-Fehler/-Dauer, Resync- und GC-Warteschlangen
VictoriaMetrics & VictoriaLogs	Ingest-Rate, Plattenbelegung, Abfragedauer, aktive Serien
zorlock (Mac mini)	mactop: Thermal State, CPU/GPU, Leistung, Temperaturen, Speicher, Lüfter; Ollama-Anfragen, -Geschwindigkeit, Ladezeiten der Modelle

Soll gegen known (NetBird): „expected“ berechnet OpenTofu aus den Policies mit der Gruppenmitgliedschaft beim letzten `just netbird-apply` (Beispiel: aldrin 6, grammi 8, cuckoosegg 4). „known“ ist, was der Management-Server tatsächlich verteilt. Ein später hinzukommendes Gerät erhöht known sofort, das Soll erst nach dem nächsten Apply + Deploy von grammi – gewollt, damit ein neuer Peer auffällt. Das Modell ignoriert Posture Checks; siehe Kopf von `expected-peers.tf`.

Grafana-Falle: State-Timelines mit `mergeValues` und Farbmodus „thresholds“ machen aus Zahlen Schwellwert-Stufen („1+“, „< 1“), bevor die Wert-Zuordnungen greifen. Farbmodus „fixed“ verwenden und Text und Farbe in die Zuordnungen legen.

Neuen Host aufnehmen

Ein Host steht an mehreren Stellen im Repo, weil NixOS und OpenTofu bewusst keine gemeinsame Host-Liste haben (siehe Was bewusst nicht verdrahtet ist). Diese Anleitung geht sie der Reihe nach durch.

1. Maschine bereitstellen

Gemietete VMs werden beim Anbieter von Hand angelegt; Bare Metal und die Subraum-VM folgen den Installationsanleitungen unter `docs/install-*.md`. Am Ende steht ein NixOS mit SSH-Host-Key – aus dem entsteht gleich der age-Schlüssel des Hosts.

2. Konfiguration anlegen

- `hosts/<host>/configuration.nix` und `hardware-configuration.nix` anlegen.
- In `flake.nix` unter `nixosConfigurations` eintragen, mit expliziter Modulliste. Was bewusst fehlt, bekommt einen Kommentar mit Begründung daneben.

3. Überall eintragen, wo Hosts aufgezählt sind

Datei	Eintrag
`flake.nix`	`nixosConfigurations.<host>`
`justfile`	`hosts := "…"` (nur Pull-Hosts, für `ship-all`)
`terraform/netbird/main.tf`	`local.hosts`: Host → Setup-Key-Gruppe (`servers` o. ä.)
`terraform/desec-tokens/main.tf`	`local.hosts`: erlaubte Quelladressen, `acme_names`
`terraform/github/main.tf`	`local.hosts`: Deploy-Key nur lesend oder mit Schreibrecht

4. sops-Empfänger

Den age-Schlüssel aus dem SSH-Host-Key des neuen Hosts ableiten (`ssh-to-age` auf `/etc/ssh/ssh_host_ed25519_key.pub`) und in `.sops.yaml` eintragen: einen Anker unter `keys` und eine `creation_rule` für `hosts/<host>/(secrets|netbird|desec|github)\.yaml$` mit dem Host und `admin` als Empfängern.

5. Schlüssel und Tokens erzeugen

Rezept	Ergebnis	Hinweis
`just netbird-key <host>`	`hosts/<host>/netbird.yaml`	Einmal-Setup-Key, **24 h gültig** – direkt danach deployen
`just desec-key <host>`	`hosts/<host>/desec.yaml`	nur für Hosts mit eigenen Zertifikaten
`just github-key <host>`	`hosts/<host>/github.yaml` + `github-deploy.pub`	nur für Pull-Hosts

Rezepte, die auf dem MacBook sops brauchen, laufen mit der 1Password-Anmeldung davor (siehe just-Rezepte).

6. Deployen und prüfen

- Erster Deploy innerhalb von 24 h nach `netbird-key`. Der Host registriert sich beim Start selbst im Mesh; Tofu findet ihn über `data "netbird_peer"` am Namen.
- `just topology` ausführen und das Ergebnis committen – sonst schlägt `nix flake check` fehl.
- App-Secrets erst eintragen, wenn der Host läuft: Secrets eintragen.
- Doku nachziehen: Hosts & Rollen, Services.

Neuen Dienst veröffentlichen

Dienste heißen nach ihrer Funktion (`rezepte`, nicht `mealie`); neue Webdienste auf grammi nutzen PostgreSQL. Zuerst entscheiden, wie der Dienst erreichbar sein soll – davon hängt ab, welche Dateien sich ändern.

Erreichbarkeit	Für	Beispiele
Nur im Mesh	Admin-Oberflächen, Dienste für die eigenen Geräte	`s3-admin`, `llm`, `dokumente`
Öffentlich hinter dem Reverse Proxy mit SSO	Web-Apps, die auch von unterwegs ohne NetBird-App erreichbar sein sollen	`rezepte`, `grist`, `metrics`
Direkt öffentlich	Dienste mit eigenem Login für Maschinen oder alle	`auth`, `s3.aldrin`

Nur im Mesh

1. In `terraform/netbird/mesh-services.tf` eine Zeile unter `local.mesh_services`: Name → Host(s). Daraus entsteht eine eigene NetBird-DNS-Zone mit A-Record(s) auf die Mesh-Adresse(n); öffentliches DNS kennt den Namen nicht.
2. `just netbird-apply` – neue Zonen kommen seit NetBird 0.81 ohne Neustart bei den Clients an.
3. Auf dem Host einen nginx-vHost mit `enableACME`, `acmeRoot = null` (DNS-01 über deSEC) und `allow 192.0.2.0/24; deny all;`.
4. Steht der Dienst auf ursalia oder zorlock: den Namen in `acme_names` des Hosts in `terraform/desec-tokens/main.tf` ergänzen und `just desec-apply` – deren Tokens dürfen nur genau diese TXT-Records schreiben.
5. Host deployen, `just topology` (neuer vHost).

Öffentlich hinter dem Reverse Proxy

Für Dienste auf grammi:

1. Eintrag in `hosts/grammi/proxy-services.json`: Name → App, Port, Pocket-ID-Gruppen. Tofu (`reverse-proxy.tf`) und `topology.nix` lesen dieselbe Datei.
2. Der Dienst lauscht auf allen Adressen; öffentlich sind nur 80/443 offen, `nb-mesh` ist vertrauenswürdig.
3. In DNSControl einen CNAME `<name>` → `proxy.homelab.example` pushen – einige Minuten vor dem Apply, sonst cacht der Resolver NXDOMAIN.
4. Erst grammi deployen, dann `just netbird-apply`. Andersherum scheitert der ACME-Prefetch des Proxys und das Dashboard hängt bei „Issuing certificate…“ (Abhilfe: `systemctl restart netbird-proxy` auf grammi).

Dienste auf anderen Hosts bekommen eine eigene Ressource in `terraform/netbird/reverse-proxy.tf` mit dem Host als Peer-Ziel (Beispiel: `dokumente` auf ursalia).

Direkt öffentlich

nginx-vHost auf dem Host (ACME per DNS-01), A/AAAA-Records in DNSControl, CrowdSec liest die nginx-Logs mit. Nur für Dienste, die ohnehin eine eigene Anmeldung haben.

Danach

- Doku: Services, bei neuem Host-Verzeichnis-Inhalt auch Inhalt von hosts/<host>/.
- Dual-Stack, wo es geht (A und AAAA).

Secrets eintragen

App-Passwörter, OIDC-Clients und Ähnliches liegen in `hosts/<host>/secrets.yaml`, verschlüsselt für den Host und den Recovery-Key `admin`. Das MacBook kann diese Dateien nicht entschlüsseln – deshalb wird auf dem Host editiert (Hintergrund: Secrets, Netzwerk & Zugriff).

Ablauf

1. `just sync <host>` – sonst liegt auf dem Host ein veralteter Stand von `secrets.yaml`, und `pull-secrets` holt ihn zurück ins Repo.
2. `just secrets <host>` – öffnet sops auf dem Host (mit dessen Host-Key, ohne 1Password-Anmeldung) und holt das Ergebnis danach per `pull-secrets` ins Repo.
3. Werte immer in Anführungszeichen: `key: "wert"`, einfache Anführungszeichen, wenn der Wert `"` oder `\` enthält. Ein unquotierter Wert kann von YAML als Zahl oder Bool gelesen werden; sops-nix bricht dann `setupSecrets` ab und **kein** Secret dieser Generation wird installiert.
4. Die NixOS-Seite (`sops.secrets."<name>"`) im Host-Modul ergänzen, committen, deployen: `just ship <host> "<nachricht>"`.

Was nicht von Hand editiert wird

Datei	Erzeugt von
`hosts/<host>/netbird.yaml`	`just netbird-key <host>`
`hosts/<host>/desec.yaml`	`just desec-key <host>`
`hosts/<host>/github.yaml`	`just github-key <host>`
`terraform/*/secrets.yaml`	sops auf dem MacBook mit dem Workstation-Key aus 1Password

Alles, was auf dem MacBook sops braucht, läuft mit vorangestellter Anmeldung – siehe just-Rezepte.

Schlüssel rotieren

Was sich wie erneuern lässt und was danach zu tun ist. Die Rezepte ersetzen den alten Schlüssel jeweils sofort – deshalb direkt danach committen und deployen.

Schlüssel	Rezept	Danach	Wann
NetBird-Setup-Key eines Hosts	`just netbird-key <host>`	Deploy innerhalb von 24 h	bei Neuinstallation oder verlorenem Client-State
deSEC-Token eines Hosts	`just desec-key <host>`	sofort deployen, dann `just desec-verify`	bei Verdacht, sonst nie
GitHub-Deploy-Key eines Hosts	`just github-key <host>`	commit + deploy	bei Verdacht

Schlüssel	Rezept	Danach	Wann
Tofu-Token für NetBird (Service-User `terraform`)	`just netbird-renew-token`	–	jährlich
App-Secrets	`just secrets <host>` (siehe Secrets eintragen)	deploy	bei Verdacht

SSH-Host-Key (= sops-Empfänger)

Der age-Schlüssel eines Hosts ist aus seinem SSH-Host-Key abgeleitet. Ein neuer Host-Key heißt deshalb auch:

1. Neuen age-Schlüssel per `ssh-to-age` ableiten und in `.sops.yaml` ersetzen.
2. Die Dateien des Hosts mit `sops updatekeys` neu verschlüsseln – dafür braucht es einen Schlüssel, der sie noch lesen kann (der alte Host-Key oder `admin`).
3. Committen und deployen, bevor der alte Key verschwindet.

Nach einer Rotation

- Alte Schlüssel beim Gegenüber löschen (GitHub, deSEC und NetBird erledigt das Rezept per Tofu).
- `just docs`, falls sich Namen oder Abläufe geändert haben.

Repository-Struktur

Ein Flake, `nixosConfigurations` für fünf Hosts plus `darwinConfigurations.zorlock`. Jeder Host bekommt eine explizite Modulliste in `flake.nix`; was fehlt, fehlt mit Begründung im Kommentar daneben.

Pfad	Inhalt
`flake.nix` / `flake.lock`	Inputs, Host-Definitionen, Pakete, `lib.topology`, `checks`
`modules/`	17 geteilte Module: base, nix, ssh, firewall, networking (ifstate), acme, crowdsec, tailscale, netbird, metrics/metrics-shipping, logging/log-shipping, qemu-guest, deploy, auto-upgrade-git, docs-site
`hosts/<host>/`	`configuration.nix`, Hardware, Dienst-Module, sops-Dateien (`secrets.yaml`, `netbird.yaml`, `desec.yaml`)
`pkgs/`	Eigene Pakete: garage-ui, logchef, paperless-gpt, prometheus-paperless-exporter, ollama-metrics-proxy
`terraform/netbird/`	OpenTofu: NetBird-Account, Policies, DNS-Zonen, Reverse-Proxy-Dienste, Personen/Gruppen (inkl. Pocket ID)
`terraform/desec-tokens/`	OpenTofu: eng gescopte deSEC-Tokens pro Host für DNS-01
`terraform/github/`	OpenTofu: die Repos `nixlab` (privat) und `nixlab-docs` (öffentlich), Deploy-Keys pro Host
`topology.nix`	Generiert das Topologie-Dashboard für Grafana aus den Host-Configs
`justfile`	Alle Bedienabläufe: deploy, secrets, netbird-*, desec-*
`docs/`	Diese Projektdokumentation: Seiten unter `site/{de,en}/`, Zensical-Konfiguration, Build (`build.sh`, `site-src.sh`, `assemble.py`, `gen-reference.py`), Installationsanleitungen (ursalia, cuckoosegg), Upstream-Issue-Entwürfe
`dnsconfig.js`	Symlink ins separate DNSControl-Repo – öffentliche DNS-Records leben dort
`.sops.yaml`	age-Empfänger pro Host, `admin` (Offline-Recovery) und `workstation` (1Password)

Wo ändere ich was?

Welche Datei für welche Änderung zuständig ist, wie oft das vorkommt und was danach zu tun ist.

Was	Wo	Wie oft	Danach
Dienst, Paket oder Option auf einem Host	`hosts/<host>/*.nix`	häufig	`just ship <host> "…"`
Etwas für alle Hosts	`modules/*.nix`	gelegentlich	`just ship-all "…"`; neues Modul: öffentlich oder in `docs/modules-private.txt`?
Name hinter dem Reverse Proxy (grammi)	`hosts/grammi/proxy-services.json`	selten	CNAME in DNSControl, grammi deployen, `just netbird-apply`
Name nur im Mesh	`terraform/netbird/mesh-services.tf`	selten	`just netbird-apply`
NetBird-Policies, Gruppen, Personen, Routen	`terraform/netbird/*.tf`	selten	`just netbird-plan`, `just netbird-apply`
Öffentliche DNS-Records	DNSControl-Repo (`dnsconfig.js`)	selten	dort pushen
Was ein deSEC-Token darf	`terraform/desec-tokens/main.tf`	selten	`just desec-apply`
Repos, Deploy-Keys	`terraform/github/`	selten	`just github-plan`, `just github-apply`
Version eines automatisch aktualisierten Inputs	`flake.lock`	nachts automatisch	–
Version eines separat gepinnten Inputs	`flake.lock` (`nix flake update <input>`)	von Hand	Hosts deployen, die ihn nutzen
Grafana-Dashboard	`hosts/grammi/dashboards/*.json`	gelegentlich	grammi deployen
Topologie (neuer Host, Exporter, vHost)	`topology.nix` erzeugt sie	bei Bedarf	`just topology`, committen
Secrets eines Hosts	`hosts/<host>/secrets.yaml`	selten	Secrets eintragen
Diese Dokumentation	`docs/site/{de,en}/`	mit jeder Änderung	`just docs`

Inhalt von hosts/<host>/

Was in einem Host-Verzeichnis liegt und wer es schreibt. Nicht jeder Host hat jede Datei.

Datei	Inhalt	Geschrieben von
`configuration.nix`	Host-Konfiguration: Netz, Dienste, vHosts	von Hand
`hardware-configuration.nix`	Hardware, Dateisysteme	`nixos-generate-config`, danach von Hand
`disk-config.nix`	Partitionierung (disko, nur ursalia)	von Hand
weitere `*.nix`	ein Dienst pro Datei, z. B. `backup.nix`, `samba.nix`, `netbird.nix`, `traefik.nix`	von Hand
`secrets.yaml`	App-Secrets, verschlüsselt für Host + admin	`just secrets <host>`

Datei	Inhalt	Geschrieben von
`netbird.yaml`	NetBird-Setup-Key	`just netbird-key <host>` (Tofu)
`desec.yaml`	deSEC-Token für DNS-01	`just desec-key <host>` (Tofu)
`github.yaml`	privater Teil des Deploy-Keys	`just github-key <host>`
`github-deploy.pub`	öffentlicher Teil, von `terraform/github` gelesen	`just github-key <host>`
weitere `*.yaml`	sops-Dateien einzelner Dienste, z. B. `netbird-server.yaml`, `netbird-proxy.yaml` (grammi), `push-cuckoosegg.yaml` (ursalia)	ihre Rezepte, z. B. `just netbird-server-secrets`, `just netbird-proxy-token`
`push-key.pub`	öffentlicher Schlüssel für den Push-Deploy (nur cuckoosegg)	`just cuckoosegg-push-key`
`dashboards/*.json`	Grafana-Dashboards (nur grammi)	von Hand, `topology.json` per `just topology`
`proxy-services.json`	Dienste hinter dem Reverse Proxy (nur grammi)	von Hand, gelesen von Tofu und `topology.nix`

Inputs und ihre Pins

Nur `nixpkgs`, `nixpkgs-unstable` und `sops-nix` laufen im nächtlichen Auto-Upgrade mit (`updateInputs` in `modules/deploy.nix`). Alles, was bei einem kaputten Release das ganze Lab aussperren würde, ist separat gepinnt und wird von Hand gebumpt.

Input	Branch / Tag	Wer nutzt es	Nachts mit?
`nixpkgs`	nixos-26.05	alle NixOS-Hosts	ja
`nixpkgs-unstable`	nixos-unstable	grammi (Grist), ursalia (Paperless 3.x), zorlock (mactop)	ja
`sops-nix`	main	alle	ja
`nixpkgs-netbird`	nixos-unstable	NetBird-Server und alle Clients	nein – Server zuerst, dann Clients
`nixpkgs-pocketid`	nixos-26.05	zummi (pocket-id-Paket)	nein
`disko`	main	ursalia (alle Mounts)	nein
`lanzaboote`	v1.1.0	ursalia (Secure Boot)	nein
`nixpkgs-darwin`, `nix-darwin`	26.05-darwin	zorlock	nein (kein Auto-Upgrade)

`nix flake check` prüft zusätzlich, dass `hosts/grammi/dashboards/topology.json` zu dem passt, was `topology.nix` gerade erzeugt – ein neuer Exporter ohne `just topology` lässt den Check fehlschlagen.

Hosts & Rollen

Drei gemietete VMs tragen die öffentlichen Dienste, ursalia und zorlock stehen zu Hause, cuckoosegg im Subraum. Alle sind über `<host>.peer.homelab.example` im NetBird-Mesh erreichbar; root-SSH gibt es nur dort.

Host	Plattform	Rolle	Besonderheiten
grammi	NixOS, gemietete VM	NetBird-Server (`mesh.homelab.example`) + Reverse Proxy, Traefik, Monitoring (VictoriaMetrics/-Logs, Grafana, logchef), Mealie, Grist, PostgreSQL	Zentrale Senke für Metriken und Logs aller Hosts
zummi	NixOS, gemietete VM	Pocket ID (`auth.homelab.example`) – Login für alle Dienste und NetBird	pocket-id separat gepinnt
aldrin	NixOS, gemietete VM	Garage S3 (`s3.aldrin.homelab.example`), garage-ui (`s3-admin`), Backup-Ziel für restic, Doku-Site (`doku`)	Fork-Paket garage-ui mit Subpath-Support
ursalia	NixOS, Bare Metal (ThinkCentre M720q)	Paperless-ngx 3 + paperless-gpt (`dokumente`), Doku-Site (`doku`), Samba für den Scanner, Routing-Peer ins Heim-LAN, Build-Host für cuckoosegg	LUKS + btrfs via disko, TPM-Unlock, Secure Boot (lanzaboote), WLAN (Intel BE200), kein CrowdSec
cuckoosegg	NixOS, VM auf dem Subraum-Proxmox	NetBird-Routing-Peer in die Subraum-Netze	Gruppe `subraum-devices` statt `servers`, kein Repo/Deploy-Key, wird per Push deployed
zorlock	macOS + nix-darwin, Mac mini M4	Ollama auf der Apple-GPU (`llm`), genutzt von paperless-gpt	Keins der `modules/*.nix`; FileVault-Unlock nach Reboot per LAN-SSH

Services

Dienste sind nach Funktion benannt, nicht nach Produkt. Es gibt drei Arten der Erreichbarkeit: direkt öffentlich, öffentlich hinter dem NetBird-Reverse-Proxy mit SSO, oder nur im Mesh.

Name (`.homelab.example`)	App	Host	Erreichbar
`auth`	Pocket ID	zummi	öffentlich (nginx)
`mesh`	NetBird Management/Dashboard	grammi	öffentlich (Traefik terminiert TLS)
`s3.aldrin`	Garage S3-API	aldrin	öffentlich (nginx)
`rezepte`	Mealie	grammi	Reverse Proxy + SSO
`grist`	Grist	grammi	Reverse Proxy + SSO
`logs`	logchef	grammi	Reverse Proxy + SSO
`metrics`	Grafana	grammi	Reverse Proxy + SSO
`vm` / `vl`	VictoriaMetrics / VictoriaLogs UI	grammi	Reverse Proxy + SSO (über Traefik-Entrypoint `apps`, Port 8090)
`s3-admin`	garage-ui	aldrin	nur Mesh
`dokumente`	Paperless + paperless-gpt (`/paperless-gpt/`)	ursalia	nur Mesh; öffentlicher Zugang über den Proxy in Arbeit
`llm`	Ollama (hinter Caddy)	zorlock	nur Mesh
`doku`	Diese Dokumentation (private Fassung)	aldrin + ursalia	nur Mesh, ein A-Record je Host

Reverse Proxy: Traefik auf grammi besitzt Port 443. `mesh.homelab.example` terminiert es selbst, alles andere reicht es per TCP-Passthrough mit PROXY-Protokoll v2 an `netbird-proxy` weiter, das TLS und ACME

(tls-alpn-01) selbst macht. Login am Proxy ist SSO gegen Pocket ID, begrenzt auf die NetBird-Gruppen der jeweiligen App.

Mesh-only-Namen: Jeder Name ist eine eigene NetBird-DNS-Zone mit einem A-Record auf die Mesh-Adresse (192.0.2.x). Öffentliches DNS kennt sie nicht; das Zertifikat kommt per DNS-01 über deSEC.

Querschnitt auf allen Linux-Hosts: vmagent schickt Metriken, Vector schickt journald- und nginx-Logs an grammi. CrowdSec liest die nginx/Traefik-Logs auf den öffentlichen Hosts. restic-Backups gehen in Garage-Buckets auf aldrin, für Paperless zusätzlich append-only zu BorgBase.

Module

Die 17 Module unter `modules/` tragen die gemeinsame Konfiguration aller Hosts; jede Datei erklärt im Kopfkommentar, warum sie so ist. Die öffentliche Fassung enthält sie anonymisiert unter `modules/` – als Referenz, nicht direkt importierbar: einige lesen `hosts/<host>/*.yaml` und setzen sops-nix voraus. Vier haben eigene Optionen.

Modul	Zweck	Optionen / Besonderheiten
auto-upgrade-git.nix	Nächtliches `nixos-rebuild switch`, committet und pusht die neue `flake.lock` zurück ins Repo	`services.autoUpgradeGit`: `enable`, `flakeDir`, `flakeAttr`, `updateInputs`, `dates`, `allowReboot`, `rebootWindow`, `deployKeySecret`, `gitAuthorEmail`
log-shipping.nix	journald- und nginx-Logs per Vector an VictoriaLogs	`services.logShipping`: `enable`, `victoriaLogsUrl`
metrics-shipping.nix	vmagent scrapt die lokalen Exporter und schreibt per remote_write an VictoriaMetrics	`services.metricsShipping`: `enable`, `remoteWriteUrl`, `exporterPorts`
deploy.nix	Die Lab-Policy für auto-upgrade-git: GitHub-Deploy-Key, gepinnter Hostkey, Reboot-Fenster 04:00–06:00	getrennt vom Mechanismus, damit der wiederverwendbar bleibt
acme.nix	ACME-Defaults für jeden TLS-Host: DNS-01 gegen deSEC	Token aus `hosts/<host>/desec.yaml` (von Tofu)
netbird.nix	NetBird-Client im Mesh, Registrierung per Setup-Key, lokale Metriken	Setup-Key aus `hosts/<host>/netbird.yaml` (von Tofu)
tailscale.nix	Gemeinsamer Tailscale-Agent	`tailscale serve` bleibt in der Host-Config
ssh.nix	Wer sich von wo anmelden darf: root nur über Mesh oder Tailscale	Schlüssel bleiben pro Host
firewall.nix	Öffentlich nur 22/80/443, Tailnet als vertrauenswürdiges Interface	
crowdsec.nix	CrowdSec-Engine und Firewall-Bouncer, Whitelist für die Lab-Hosts	Umgehungen für Fehler in den nixpkgs-Modulen, im Kopf dokumentiert
networking.nix	ifstate-Gerüst, systemd-resolved mit DNS-over-TLS, keine Fallback-Resolver	Adressen und Routen bleiben in der Host-Config
metrics.nix	Prometheus-Exporter pro Host	Schutz über die Firewall statt über Bind-Adressen
logging.nix	Begrenzt journald, das sonst bis 4 GB wächst	
qemu-guest.nix	QEMU Guest Agent mit gesperrter RPC-Liste: Snapshots ja, Shell und Dateizugriff über den Kanal nein	
nix.nix	Nix-Daemon, Garbage Collection, Store-Optimierung	

Modul	Zweck	Optionen / Besonderheiten
docs-site.nix	Die private Doku-Site (Flake-Paket `docs-site`) unter `doku` im Mesh	`services.docsSite`: `enable`, `package`, `fqdn`; auf aldrin und ursalia
base.nix	Locale, Konsole, Zeit und Werkzeuge zum Debuggen	

just-Rezepte

Rezept	Zweck
`just check <host>`	`git pull` + `nixos-rebuild dry-activate`
`just secrets <host>` / `pull-secrets <host>`	Host-Secrets auf dem Host editieren und zurückholen
`just topology`	Topologie-Dashboard neu erzeugen (sonst schlägt `nix flake check` fehl)
`just netbird-plan` / `netbird-apply`	NetBird-Account ändern; apply schreibt auch `expected-peers.json` neu (committen, grammi deployen)
`just netbird-key <host>`	Neuer Setup-Key für einen Host, 24 h Zeit zum Deployen
`just netbird-import`	Verlorenen Tofu-State aus dem Account wiederherstellen
`just netbird-renew-token`	Jährliche Erneuerung des Service-User-Tokens
`just github-key <host>`	Neuer GitHub-Deploy-Key für einen Host (Schlüssel per sops, Registrierung per Tofu)
`just github-plan` / `github-apply`	Repos und Deploy-Keys auf GitHub ändern
`just desec-key <host>` / `desec-verify`	deSEC-Token tauschen bzw. Scopes von jedem Host aus prüfen
`just gc <host>`	Store-Generationen älter als 7 Tage löschen
`just docs`	Diese Doku bauen: private PDFs (de/en) und die anonymisierte öffentliche Fassung (README, PDFs, Site-Quellen, Module) in `public-docs/` (Clone von `nixlab-docs`, gitignored), dort committet und gepusht; GitHub baut daraus die Site
`just docs-serve`	Private Site mit echten Werten lokal bauen und unter `http://127.0.0.1:8000` zeigen

Rezepte, die sops auf dem MacBook brauchen, laufen mit vorangestellter Anmeldung bei 1Password, die den age-Key über `SOPS_AGE_KEY_CMD` bereitstellt:

```
eval $(op signin) && export SOPS_AGE_KEY_CMD='op read "op://<vault>/<item>/password"' && just netbird-plan
```

Troubleshooting

- Nach einem `nixos-rebuild` sagt `systemctl --failed` nichts über die Vergangenheit – der Switch räumt Failed-States ab. Stattdessen `journalctl -u <unit>`.
- NetBird down, Tailscale noch da: `NIXLAB_DOMAIN=example-tailnet.ts.net` vor das Rezept setzen.
- Tofu-Rezepte verweigern den Lauf gegen leeren State, damit keine zweite `servers`-Gruppe oder Token-Garnitur entsteht.
- WLAN auf ursalia: `iw dev wlp2s0f0 link` zeigt Band, Signal und PHY-Modus. Das 5-GHz-Netz des Routers liegt auf einem DFS-Kanal mit 300 TU Beacon-Intervall, ein einzelner Scan kann es verpassen; Band, Signal, Raten und PHY-Modus gehen auch an VictoriaMetrics (node_exporter-Collector `wifi` plus `wifi_link_*`), Grafana-Dashboard „Homelab / WLAN“.
- Nach einem Netzwechsel kann NetBird minutenlang an einem toten P2P-Weg festhalten und trotzdem `Connected` zeigen (gesehen auf ursalia, 2026-10-19). Abhilfe: `systemctl restart netbird-mesh` auf dem Host, notfalls über Tailscale.

Konventionen

- Pro Thema ein eigener Git-Worktree (`.worktrees/<thema>`), nach Abschluss entfernen und Branch löschen. Tofu-State bleibt dabei immer im Haupt-Checkout.
- Erst planen, dann nach Freigabe umsetzen.
- Subdomains nach Funktion (`rezepte`, nicht `mealie`).
- Neue Webdienste auf grammi nutzen PostgreSQL statt SQLite.
- Dual-Stack, wo immer möglich.
- Kommentare im Code erklären das *Warum*, inklusive der Fehler, die zu einer Entscheidung geführt haben, mit Datum.
- Code-Kommentare, Commit-Messages und Changelogs auf Englisch; die Projektdoku gibt es auf Deutsch und Englisch.

Offene Punkte

☐ NetBird-Mesh auf Dual-Stack (IPv6-Overlay) umstellen
☐ Reverse-Proxy-Stufe 3 abschließen: Paperless öffentlich unter `dokumente` (Uploads, paperless-gpt)
☐ Upstream-PRs im NetBird-Provider (#247 `direct_upstream`/Cluster-Targets, #248 `crowdsec_mode`); danach Proxy-Targets auf grammi auf `localhost` umstellen
☐ Tailscale von den Servern entfernen, sobald nichts mehr darauf angewiesen ist
☐ Grafana-Alerting (Kontaktpunkt fehlt noch) für SMART, fwupd, Paperless, NetBird Soll ≠ known, WLAN von ursalia (nicht verbunden, hängt auf 2,4 GHz, schwaches Signal)
☐ Update-Benachrichtigung für die von Hand gepinnten Inputs (NetBird, Pocket ID)
☐ Root-Passwörter deklarativ über sops
☐ Host-Liste an einer Stelle führen statt in `flake.nix`, den Tofu-Modulen und `justfile`